МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики
Кафедра технической кибернетики

ЛАБОРАТОРНАЯ РАБОТА № 2

«Формирование отчетов в Apache Spark»

по курсу
Большие данные

Группа 6403
Студент_____ А.А. Кучумов
(*подпись*)

Преподаватель,
Ассистент каф. ТК _____ В.И. Проценко
(*подпись*)

Самара 2025

ЗАДАНИЕ

Сформировать отчёт с информацией о 10 наиболее популярных языках программирования по итогам года за период с 2010 по 2020 годы. Отчёт будет отражать динамику изменения популярности языков программирования и представлять собой набор таблиц «топ-10» для каждого года.

Получившийся отчёт сохранить в формате Apache Parquet.

Для выполнения задания вы можете использовать любую комбинацию Spark API: RDD API, Dataset API, SQL API.

ХОД ВЫПОЛНЕНИЯ

Лабораторная работа выполнялась на операционной системе Windows, а для работы Spark в этом случае необходима переменная среды «HADOOP_HOME», указывающая путь к winutils.exe и hadoop.dll, которые, в свою очередь, позволяют работать с Hadoop под Windows. С помощью модуля os была задана упомянутая выше переменная среды, а также в PATH был прописан путь к ней, что представлено на рисунке 1.

1. Импортируем модуль os для работы с операционной системой (лабораторная работа выполнялась под Windows).

```python
import os
```

✓ 0.0s Python

2. Задаем путь к директории hadoop под Windows (winutils.exe и hadoop.dll), записываем его в переменную окружения HADOOP_HOME, добавляем ее PATH, чтобы при запуске процесса была найдена hadoop.dll для работы Spark.

```python
_HADOOP_HOME = r"C:\hadoop"
os.environ["HADOOP_HOME"] = _HADOOP_HOME
os.environ["PATH"] = _HADOOP_HOME + r"\bin;" + os.environ.get("PATH", "")
```

✓ 0.0s Python

Рисунок 1 – Задание переменной среды «HADOOP_HOME»

Далее были импортированы необходимые для выполнения работы инструменты из PySpark, а также создана Spark-сессия. Проделанные действия представлены на рисунке 2.

3. Импортируем необходимые модули из pyspark.sql.

```python
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.window import Window
```

✓ 0.0s Python

4. Создаем сессию Spark для всего JupyterNotebook.

```python
spark = (
    SparkSession.builder
    .appName("6403_KuchumovAA_BigData_Lab_2")
    .master("local[*]")
    .config("spark.sql.shuffle.partitions", "4")
    .config("spark.driver.extraLibraryPath", _HADOOP_HOME + r"\bin")
    .getOrCreate()
)
```

✓ 0.6s Python

Рисунок 2 – Импорты и создание Spark-сессии

После того, как все готово к работе, я собрал данные о языках программирования из файла «programming-languages.csv», которые представляет собой набор строчек, содержащих название языка программирования и ссылку на статью о нем на ресурсе Wikipedia. С помощью

опции «header»-«true» я пропустил первую строку, которая содержала названия столбцов; далее столбец с названиями языков программирования был обработан следующим образом:

1. Убраны пробелы в начале и конце строки.
2. Строка переведена в нижний регистр.
3. Имя столбца было задано «language_name».

На рисунке 3 представлен код, выполняющий эти операции и примеры из получившейся таблицы.

```
languages_df = (
    spark.read
    .option("header", "true")
    .csv("data/programming-languages.csv")
    .select(F.lower(F.trim(F.col("name"))).alias("language_name"))
    .distinct()
)

print(languages_df.show(5))
```

✓ 0.3s Python

```
+-------------+
|language_name|
+-------------+
|      a# .net|
|          abc|
|    abc algol|
|        abset|
|     ace dasl|
+-------------+
only showing top 5 rows

None
```

Рисунок 3 – Формирование списка языков программирования

После того, как был сформирован список с названиями языков программирования, я начал обработку дампа постов с ресурса Stack Overflow. Сначала была создана таблица из «сырых» данных, а потом из нее удалялись служебные строки, а также оставались только те строки, которые представляли собой посты с вопросами. Описанные выше действия с результатами работы представлены на рисунке 4.

6. Читаем файл как текстовый в сырой DataFrame: каждая строка файла - строка таблицы с колонкой value.

```python
raw_df = spark.read.text("data/posts_sample.xml")
print(raw_df.show(5))
```

✓ 0.0s Python

```
+--------------------+
|               value|
+--------------------+
|<?xml version="1....|
|             <posts>|
|  <row Id="4" Pos...|
|  <row Id="6" Pos...|
|  <row Id="7" Pos...|
+--------------------+
only showing top 5 rows

None
```

7. Из сырого DataFrame оставляем строки, которые удовлетворяют следующим условиям:
   - содержание подстроки "<row" - отсекаем служебные строки (например, <?xml ...>)
   - содержание подстроки "PostTypeId="1"" - в дампе Stack Overflow это вопросы (ответы имеют "PostTypeId="2"")

```python
questions_rows = raw_df.filter(
    F.col("value").contains("<row") &
    F.col("value").contains('PostTypeId="1"')
)
print(questions_rows.show(5))
```

✓ 0.0s Python

```
+--------------------+
|               value|
+--------------------+
|  <row Id="4" Pos...|
|  <row Id="6" Pos...|
|  <row Id="9" Pos...|
|  <row Id="11" Po...|
|  <row Id="13" Po...|
+--------------------+
only showing top 5 rows

None
```

Рисунок 4 – Предобработка дампа постов

После первоначальной обработки я сформировал таблицу с двумя столбцами: год и теги. Оба значения извлекались из строк с помощью регулярных выражений. К значениям лет дополнительно применялся фильтр для того, чтобы задать нужный диапазон: от 2010 до 2020. На рисунке 5 представлен код и получившаяся таблица.

8. Выбираем строки с помощью select с годом и тегами, извлеченными с помощью регулярных выражений, а с помощью filter оставляем только те строки, в которых значение года находится в диапазоне от 2010 до 2020.

```python
parsed_tags_rows = questions_rows.select(
    F.regexp_extract("value", r'CreationDate="(\d{4})', 1)
        .cast("int").alias("year"),
    F.regexp_extract("value", r'Tags="([^"]*)"', 1).alias("tags"),
).filter(
    F.col("year").between(2010, 2020) & (F.col("tags") != "")
)
print(parsed_tags_rows.show(5))
```

✓ 0.0s Python

```
+----+--------------------+
|year|                tags|
+----+--------------------+
|2010|&lt;c++&gt;&lt;ch...|
|2010|&lt;sharepoint&gt;...|
|2010|&lt;iphone&gt;&lt...|
|2010|&lt;symfony1&gt;&...|
|2010|        &lt;java&gt;|
+----+--------------------+
only showing top 5 rows

None
```

Рисунок 5 – Формирование таблицы «год-теги»

Следующим шагом я добавил новый столбец в таблицу, которая получилась на предыдущем шаге, в котором теги уже были в читаемом формате, разделенные такими разделителями: «<», «>». Код, выполняющий это преобразование, и результат, который получился продемонстрированы на рисунке 6.

9. Преобразуем набор тегов в каждой строке в читаемый вид с помощью регулярных выражений.

```python
str_tags_rows = parsed_tags_rows.withColumn(
    "tags_str",
    F.regexp_replace(
        F.regexp_replace("tags", "&lt;", "<"),
        "&gt;", ">"
    )
)
print(str_tags_rows.show(5))
```

✓ 0.0s Python

```
+----+--------------------+--------------------+
|year|                tags|            tags_str|
+----+--------------------+--------------------+
|2010|&lt;c++&gt;&lt;ch...|<c++><character-e...|
|2010|&lt;sharepoint&gt;...|<sharepoint><info...|
|2010|&lt;iphone&gt;&lt...|<iphone><app-stor...|
|2010|&lt;symfony1&gt;&...|<symfony1><schema...|
|2010|        &lt;java&gt;|              <java>|
+----+--------------------+--------------------+
only showing top 5 rows

None
```

Рисунок 6 – Создание столбца с тегами в читаемом виде

Далее на базе этой таблицы была построена таблица, в которой один столбец содержал значения лет, а второй столбец содержал один конкретный тег, который был упомянут в соответствующий ему год. Решение этой задачи представлено на рисунке 7.

10. Строим новую таблицу, где каждый отдельный тег становится отдельной строкой с тем же годом.

```python
clean_tags_rows = str_tags_rows.select(
    "year",
    F.explode(
        F.split(
            F.regexp_replace("tags_str", r"^<|>$", ""),
            r"><"
        )
    ).alias("tag")
).select("year", F.lower("tag").alias("tag"))
print(clean_tags_rows.show(5))
```

✓ 0.1s Python

```
+----+-----------------+
|year|              tag|
+----+-----------------+
|2010|              c++|
|2010|character-encoding|
|2010|        sharepoint|
|2010|          infopath|
|2010|            iphone|
+----+-----------------+
only showing top 5 rows

None
```

Рисунок 7 – Формирование таблицы «год-тег»

Далее осуществлялась операция внутреннего соединения для списка языков программирования и таблицы, полученной путем обработки дампа постов, по совпадениям тегов и названий языков программирования. Описанные выше действия показаны на рисунке 8.

11. С помощью внутреннего соединения оставляем только те строки, теги в которых содержат язык программирования, а с помощью select оставляем только год и название языка программирования.

- join(...) - таблица с парами год-тег соединяется с таблицей со списком языков программирования по условию тег совпадает с языком программирования; inner: в итоговую таблицу попадают только строки, для которых нашлась пара в списке языков программирования

```python
clean_language_rows = (
    clean_tags_rows
    .join(languages_df, clean_tags_rows["tag"] == languages_df["language_name"], "inner")
    .select("year", F.col("tag").alias("language"))
)
print(clean_language_rows.show(5))
```

✓ 0.2s Python

```
+----+--------+
|year|language|
+----+--------+
|2010|    java|
|2010|     php|
|2010|    ruby|
|2010|       c|
|2010|     php|
+----+--------+
only showing top 5 rows

None
```

Рисунок 8 – Внутреннее соединение двух таблиц по тегу

После соединения двух таблиц были выполнены последовательно группировка по годам и языкам программирования и агрегация в виде подсчета количества таких строк. Результат работы эти операций представлен на рисунке 9.

12. С помощью группировки и последующей функцией агрегации данных (подсчет количества) считаем, сколько раз каждый язык программирования встретился в каждом году.
- groupBy(...) - строки с одинаковой парой значений года и названия языка программирования группируются
- agg(...) - считаем число строк в каждой группе

```python
language_counts = (
    clean_language_rows
    .groupBy("year", "language")
    .agg(F.count("*").alias("count"))
)
print(language_counts.show(5))
```

✓ 0.4s Python

```
+----+----------+-----+
|year|  language|count|
+----+----------+-----+
|2010|      java|   52|
|2010|       php|   46|
|2010|         c|   20|
|2010|    python|   26|
|2010|javascript|   44|
+----+----------+-----+
only showing top 5 rows

None
```

Рисунок 9 – Группировка и агрегация данных

В качестве предпоследних действий было создано «окно», формирующую одну партицию из строк с одним и тем же значением года, а также построена итоговая таблица в формате «год-ранг-название языка программирования-количество упоминаний». Внутри одного года строки упорядочены по убыванию счётчика, и каждой паре год – язык присваивается место в рейтинге. Все эти действия показаны на рисунке 10.

13. Задаем "окно": все строки с одним и тем же значением года образуют одну партицию.
- partitionBy(...) - окно считается отдельно внутри каждого значения года ("разбиение" таблицы на группы по годам)
- orderBy(...) - внутри одного года строки упорядочиваются по убыванию количества упоминаний языка программирования

```python
year_window = Window.partitionBy("year").orderBy(F.desc("count"))
```

✓ 0.0s Python

14. Из агрегата строим итоговую таблицу - топ языков по каждому году:
- withColumn(...) - добавляет новую колонку rank, значение считается функцией rank() по заданному окну: внутри одного года строки упорядочены по убыванию счётчика, и каждой паре год - язык присваивается место в рейтинге (с учётом ничьих у rank)
- filter(...) - оставляет только строки, где rank не больше 10
- select(...) - формирует финальный набор колонок: год, ранг, название языка программирование и количество упоминаний
- orderBy(...) - сортирует весь DataFrame: сначала по возрастанию year, при равном годе - по возрастанию rank, чтобы в выводе шли год за годом, а внутри года - места 1, 2, 3, ...

```python
result = (
    language_counts
    .withColumn("rank", F.rank().over(year_window))
    .filter(F.col("rank") <= 10)
    .select("year", "rank", "language", "count")
    .orderBy("year", "rank")
)
```

✓ 0.0s Python

Рисунок 10 – Финальная обработка и формирование итоговой таблицы

В конце работы полученная таблица была сохранена в файл формата .parquet. Это продемонстрировано на рисунке 11.

15. Записываем данные в .parquet-файл и выводим его директорию.

```python
output_path = "output/"
result.write.mode("overwrite").parquet(output_path)
print(f"\nФайл .parquet сохранен по пути - {output_path}\n")
```

✓ 0.4s Python

```
Файл .parquet сохранен по пути - output/
```

Рисунок 11 – Сохранение итоговой таблицы в файл .parquet

ЗАКЛЮЧЕНИЕ

В результате выполнения лабораторной работы я реализовал процесс анализа популярности языков программирования на основе данных ресурса Stack Overflow за период с 2010 по 2020 годы. В ходе работы были освоены основные подходы к обработке больших данных с использованием Apache Spark, включая загрузку, очистку, обработку и агрегацию данных.

Был сформирован перечень языков программирования, приведённый к единому формату, а также выполнена предобработка дампа постов, включающая фильтрацию, извлечение необходимых атрибутов и преобразование структуры данных. С использованием операций соединения, группировки и оконных функций была построена итоговая таблица, отражающая количество упоминаний каждого языка по годам и его позицию в рейтинге популярности.

Полученный отчет представляет собой набор таблиц «топ-10» языков программирования для каждого года, что позволяет наглядно проследить динамику изменения их популярности в исследуемый период. Итоговые данные были сохранены в формате Apache Parquet, что обеспечивает эффективное хранение и последующую обработку.